

DM-J4310-2EC V1.1 – Motor Notes

Communication, Control Modes, Code Audit

1 Motor Calibration – Setup and Verification

Correct operation requires three calibration layers applied in order. An error in any layer is the most common source of “wrong position”, “motor does not move”, or “feedback always zero”.

1.1 The three calibration layers

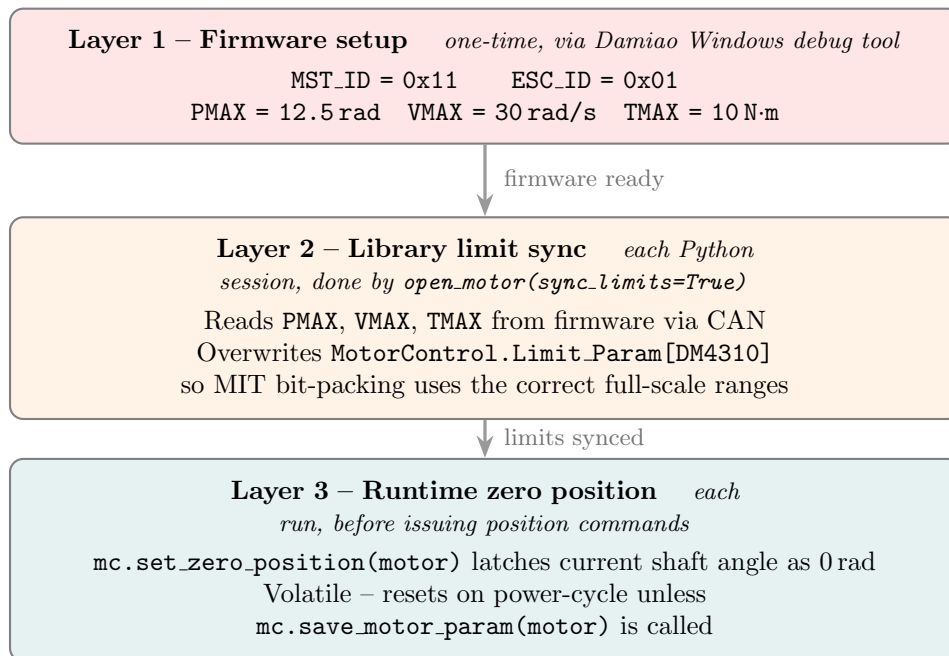


Figure 1: Three calibration layers required before commanding the motor. All three must be correct; each layer depends on the one above it.

1.2 Layer 1: Firmware parameter setup

Use the **Damiao Windows debug tool** connected via USB (Write Param tab):

Parameter	Required value	Effect if wrong	How to fix
MST_ID	0x11 (not 0x00!)	All feedback dropped; <code>getPosition()</code> returns 0	Write Param
ESC_ID	0x01 (default)	Python driver cannot address motor	Write Param
PMAx	12.5 rad	Position commands silently rescaled	Verify/adjust
VMAx	30 rad/s	Velocity commands silently rescaled	Verify/adjust
TMAx	10 N·m	Torque commands silently rescaled	Verify/adjust

Dump all current firmware values:

```
python tests/01_read_params.py
```

Fix MST_ID via Python if the debug tool is unavailable:

```
# tests/02_write_params.py -- set at top, run once
PARAM    = DM_variable.MST_ID
VALUE    = 0x11
PERSIST  = True    # calls save_motor_param() to survive power cycle
```

Why MST_ID = 0x11 is critical

The motor embeds its MST_ID as the CAN ID of every feedback frame. The Python driver accepts only frames whose CAN ID matches the `MasterID` argument passed to `Motor(..., MasterID=0x11)`.

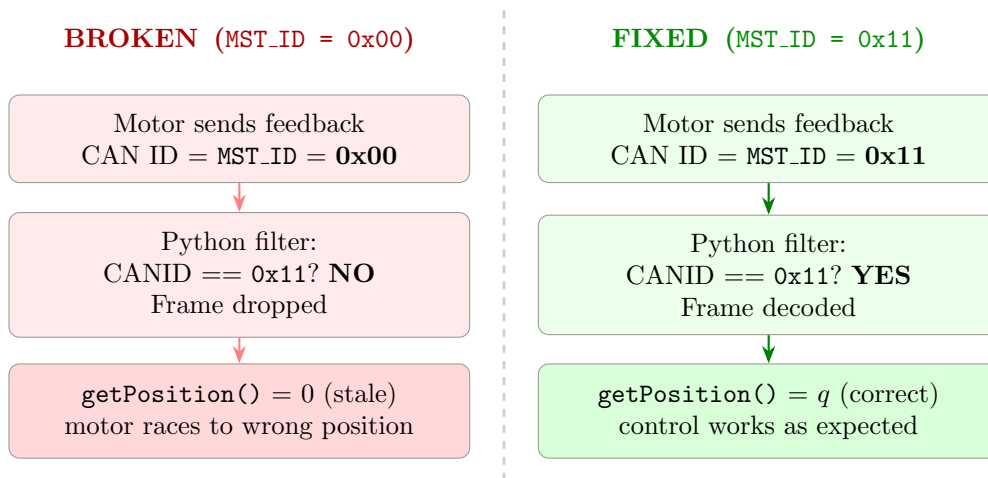


Figure 2: Effect of `MST_ID` on the Python feedback filter. The factory default 0x00 causes all feedback to be silently discarded, making `getPosition()` always return 0.

1.3 Layer 2: Library limit synchronisation

The MIT bit-packing formula maps a physical value q to an integer using P_{MAX} as the full-scale range (same principle for V_{MAX} and T_{MAX}):

$$\text{uint16} = \frac{q + P_{MAX}}{2 P_{MAX}} (2^{16} - 1)$$

If `Limit.Param` disagrees with the firmware's stored values, every MIT command is rescaled by the ratio $P_{MAX}^{\text{lib}}/P_{MAX}^{\text{fw}}$.

`open_motor(sync_limits=True)` (the default) handles this automatically:

```
pmax = mc.read_motor_param(motor, DM_variable.PMAX)
vmax = mc.read_motor_param(motor, DM_variable.VMAX)
tmax = mc.read_motor_param(motor, DM_variable.TMAX)
mc.Limit_Param[int(DM_Motor_Type.DM4310)] = [pmax, vmax, tmax]
# For this unit: PMAX=12.5, VMAX=30, TMAX=10
```

Risk: if `Limit_Param PMAX` $\neq$ firmware `PMAX`,
position error $\propto P_{MAX}^{lib}/P_{MAX}^{fw}$. Always call
`open_motor(sync_limits=True)` before MIT commands.

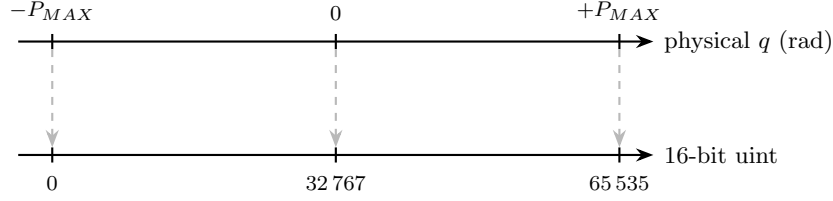


Figure 3: Linear quantisation of position for the MIT CAN frame. Velocity uses 12-bit with $\pm V_{MAX}$; torque uses 12-bit with $\pm T_{MAX}$. `PMAX` / `VMAX` / `TMAX` must match firmware exactly.

1.4 Layer 3: Runtime zero position

The output-shaft encoder is absolute single-turn (14-bit), but the *origin* (what reads as 0 rad) is a writeable firmware register. Call `set_zero_position` once after enabling to make the current physical angle the reference for all subsequent commands.

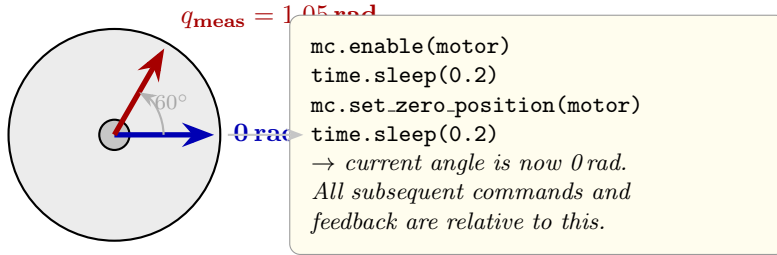


Figure 4: `set_zero_position` sends the CAN frame `[0xFF ... 0xFE]` which latches the current output-shaft encoder reading as 0 rad. The change is volatile unless `save_motor_param()` is called.

```
mc.switchControlMode(motor, Control_Type.MIT)
mc.enable(motor)
time.sleep(0.2)
mc.set_zero_position(motor)      # current angle -> 0 rad (volatile)
time.sleep(0.2)                # firmware needs ~200 ms to commit
# Now all position commands are offsets from this reference
mc.controlMIT(motor, 30.0, 1.0, math.pi/2, 0.0, 0.0) # +90 deg
```

1.5 Verification checklist

1. Power on motor (24 V). Connect HDSC USB-to-CAN adapter.
2. Open Damiao Windows debug tool, click *Read Param*. Confirm `MST_ID` = 0x11 and `ESC_ID` = 0x01. If `MST_ID` = 0x00: change to 0x11, click *Write Param*.
3. Confirm `PMAX` = 12.5, `VMAX` = 30, `TMAX` = 10. Adjust if different.
4. Run `python tests/01_read_params.py` – should print all firmware registers without timeout warnings.

- Run `python tests/03_pos_vel_control.py` – motor should move to $+90^\circ$, hold 4s, return to 0° . Any other angle indicates a PMAX mismatch (Layer 2) or an incorrect gear-ratio factor.

2 LeRobot-style Bus Architecture (multi-motor)

For robots with multiple joints (e.g. two Damiao + two CubeMars motors), a *declarative config + named bus* pattern cleanly separates hardware wiring from control logic – following the same contract as `lerobot.common.robot_utils.MotorsBus`.

2.1 Class relationships

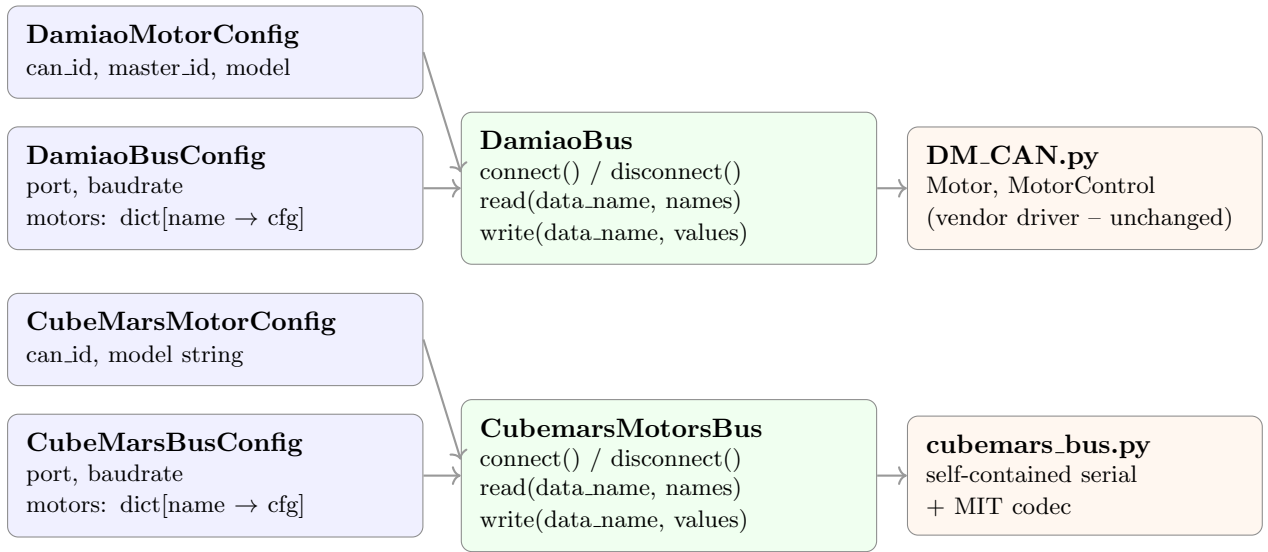


Figure 5: Separation of concerns. Config dataclasses hold hardware wiring (no driver import needed). Bus classes wrap vendor drivers behind a uniform `read/write` API.

2.2 Supported data_name values

Direction	data_name	Unit	MIT behaviour
read	position	rad	Decoded from 16-bit POS field
read	velocity	rad/s	Decoded from 12-bit VEL field
read	torque	N·m	Decoded from 12-bit TAU field
write	goal_position	rad	<code>controlMIT(K_p, K_d, q, 0, 0)</code>
write	goal_velocity	rad/s	<code>controlMIT(0, K_d, 0, v, 0)</code>
write	goal_torque	N·m	<code>controlMIT(0, 0, 0, 0, τ)</code>
write	mit_command	rad	<code>controlMIT(K_p, K_d, q, $\dot{q}$, τ_H)</code>

2.3 Protocol differences: Damiao vs. CubeMars

Property	Damiao DM-J4310-2EC	CubeMars AK-series
MIT frame	identical 8-byte bit-pack	identical 8-byte bit-pack
Feedback CAN ID	MST_ID (0x11)	ESC_ID (motor's own ID)
Limits source	read from firmware at startup	CUBEMARS_LIMITS dict
Param read/write	yes (read_motor_param)	no API available
Master ID required	yes (must be 0x11)	no
HDSC serial envelope	30-byte tx, 16-byte rx	same

2.4 Complete 4-motor example (2 Damiao + 2 CubeMars)

```
# robot_arm.py -- 4-DOF arm: shoulder+elbow (Damiao), wrist+gripper (
    CubeMars)
import time, math
from motor_config import (
    DamiaoBusConfig, DamiaoMotorConfig,
    CubeMarsBusConfig, CubeMarsMotorConfig,
)
from damiao_bus import DamiaoBus
from cubemars_bus import CubemarsMotorsBus

# --- Declare hardware wiring (edit to match your setup)
# -----
arm_cfg = DamiaoBusConfig(
    port="/dev/ttyACM0",
    motors={
        "shoulder": DamiaoMotorConfig(can_id=0x01, master_id=0x11),
        "elbow":     DamiaoMotorConfig(can_id=0x02, master_id=0x11),
    },
)
wrist_cfg = CubeMarsBusConfig(
    port="/dev/ttyACM1",
    motors={
        "wrist":     CubeMarsMotorConfig(can_id=0x01, model="AK80-9"),
        "gripper":   CubeMarsMotorConfig(can_id=0x02, model="AK60-6"),
    },
)

arm = DamiaoBus(arm_cfg)
wrist = CubemarsMotorsBus(wrist_cfg)

arm.connect(sync_limits=True, enable=True, set_zero=True)
wrist.connect(enable=True, set_zero=True)

try:
    t0 = time.time()
    while time.time() - t0 < 10.0:          # run for 10 s
        # --- read all joints
        # -----
        q_arm = arm.read("position")        # {"shoulder": rad, "elbow
            ": rad}
        q_wrist = wrist.read("position")    # {"wrist": rad, "gripper":
            rad}

        # --- sinusoidal demo trajectory
        # -----
        t = time.time() - t0
        arm.write("goal_position", {
            "shoulder": 0.5 * math.sin(t),
            "elbow":    0.3 * math.sin(2 * t),
        }, kp=30.0, kd=1.0)

        wrist.write("goal_position", {
            "wrist":    0.2 * math.sin(t + 1.0),
            "gripper":  0.1 * math.sin(t),
        }, kp=20.0, kd=0.8)
```

```

        time.sleep(0.01)          # 100 Hz control loop

finally:
    arm.disconnect()
    wrist.disconnect()

```

2.5 New source files

File	Purpose
src/motor_config.py	Dataclasses: MotorConfig, BusConfig for both vendors
src/damiao_bus.py	DamiaoBus (wraps DM_CAN.py)
src/cubemars_bus.py	CubemarsMotorsBus (self-contained)
src/DM_CAN.py	Vendor driver – unchanged
src/_common.py	Single-motor test helper – unchanged

3 Hardware summary (from PDF)

- Model: DM-J4310-2EC V1.1, 24 V geared servo motor with integrated driver.
- Gear ratio: 10:1 (output shaft).
- Rated / peak torque: 3 N·m / 7 N·m.
- Rated speed: 120 rpm; no-load max: 200 rpm ≈ 20.94 rad/s.
- Pole pairs: 14; phase $L = 340 \mu\text{H}$, $R = 650 \text{ m}\Omega$.
- Encoders: dual 14-bit magnetic, single-turn absolute on output shaft.
- CAN: 1 Mbps, standard frame; UART debug: 921600 bps.
- Recommended supply 15–32 V; over-current limit ≤ 9.8 A.
- Status LED: green on = enabled; red on = disabled; red blink = fault (8 OV, 9 UV, A OC, B MOS over-temp, C coil over-temp, D comm loss, E overload).

4 CAN frame protocol (PDF, section “Usage”)

All frames are CAN 2.0A standard, 1 Mbps. ID below is the motor’s CAN slave ID (ESC_ID, default 0x01).

4.1 Feedback frame (Master ID, default 0)

Always 8 bytes, identical across modes:

D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
ID (ERR $\ll$ 4)	POS[15:8]	POS[7:0]	VEL[11:4]	VEL[3:0] T[11:8]	T[7:0]	T_MOS	T_Rotor

POS is 16-bit unsigned mapped linearly to $[-P_{MAX}, +P_{MAX}]$ rad; VEL and T are 12-bit unsigned mapped to $[-V_{MAX}, +V_{MAX}]$ and $[-T_{MAX}, +T_{MAX}]$.

4.2 MIT mode – CAN ID = ESC_ID

8 bytes, bit-packed:

D[0]	D[1]	D[2]	D[3]	D[4]	D[5]	D[6]	D[7]
p[15:8]	p[7:0]	v[11:4]	v[3:0] Kp[11:8]	Kp[7:0]	Kd[11:4]	Kd[3:0] t[11:8]	t[7:0]

Ranges: $K_p \in [0, 500]$, $K_d \in [0, 5]$, p, v, t scaled to motor's $P_{MAX}, V_{MAX}, T_{MAX}$. *Do not set $K_d = 0$ when commanding position.*

4.3 Position–Speed mode – CAN ID = 0x100 + ESC_ID

8 bytes, two raw little-endian float32s: bytes 0–3 = `p_des` (rad), bytes 4–7 = `v_des` (rad/s, max trapezoidal speed).

4.4 Speed mode – CAN ID = 0x200 + ESC_ID

4 bytes: little-endian float32 `v_des` (rad/s).

4.5 Enable / disable / set-zero special frames

8-byte command: 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF 0xFF CMD where CMD = 0xFC enable, 0xFD disable, 0xFE set-zero.

5 How the Python library maps to the protocol

File: DM_CAN.py.

- `controlMIT(motor, kp, kd, q, dq, tau)` – packs the MIT frame exactly as the PDF table; uses `Limit_Param[MotorType]` for the $\pm P_{MAX}, \pm V_{MAX}, \pm T_{MAX}$ scaling. **This is the critical point:** the scaling *must* match the firmware-side $P_{MAX}/V_{MAX}/T_{MAX}$, otherwise the commanded position is silently rescaled.
- `control_Pos_Vel(motor, p, v)` – sends 0x100+ID with two float32s, matches PDF.
- `control_Vel(motor, v)` – sends 0x200+ID with one float32, matches PDF.
- `enable/disable/set_zero_position` – send the 0xFF..., 0xFC/0xFD/0xFE special commands.
- `switchControlMode, read/change/save_motor_param` – UART-side meta commands handled by the USB-to-CAN board.

5.1 Limit_Param table for DM4310

DM4310 : $P_{MAX} = 12.5$ rad, $V_{MAX} = 30$ rad/s, $T_{MAX} = 10$ N.m

There is no dedicated `DM_Motor_Type.DM_J4310_2EC` entry. The student's `library_test.py` therefore selects `DM_Motor_Type.DM4310` for a J-series geared motor, which is the source of the symptom described.

6 Audit of student's run – “it went to another position”

Symptom: a position command drove the shaft to a different position than expected.

6.1 Actual firmware values (read from the unit)

The Damiao Windows debug tool reports for this specific motor:

```
PMAX = 12.5 rad    VMAX = 30 rad/s    TMAX = 10 N.m
GR (Gear Ratio)      = 10
Gear factor (firmware) = 1
CAN_ID = 0x01      Master_ID = 0x00    <-- WRONG
Control mode = 1 : MIT
```

So $P_{MAX}/V_{MAX}/T_{MAX}$ already match the library's `DM_Motor_Type.DM4310` defaults. The earlier P_{MAX} -mismatch hypothesis does *not* apply here.

6.2 Real root cause 1 – the 10:1 gearbox is not compensated

With `Gear factor = 1`, the firmware does **not** divide the commanded p_{des} by the gear ratio. CAN-side commands are interpreted on the *rotor* side, so a command of $q = 8$ rad on the rotor produces only $8/10 = 0.8$ rad on the output shaft. To move the output shaft by θ rad, send 10θ rad on CAN.

6.3 Real root cause 2 – Master ID = 0x00

The firmware sends feedback with `Master_ID` as the CAN ID. The Python `Motor` object filters incoming frames against the `MasterID` passed at construction (the example uses `0x11`). With firmware set to `0x00`, the library's `recv()` drops the feedback and `getPosition()/getVelocity()/getTorque()` stay stale. Seede and the Damiao docs both warn: *do not leave Master ID at 0*. Set it to `CAN_ID + 0x10` via the debug tool, then *Write Param*.

6.4 Earlier general note – P_{MAX} mismatch on other firmware

Some J-series units ship with a smaller P_{MAX} (e.g. π rad). For those, the 16-bit POS field is decoded as

$$q_{\text{actual}} = \frac{\text{POS}}{2^{16} - 1} 2P_{MAX}^{\text{firmware}} - P_{MAX}^{\text{firmware}},$$

so a Python-side encode using $P_{MAX}^{\text{lib}} = 12.5$ silently rescales the command. Always read `PMAX`, `VMAX`, `TMAX` on start-up and overwrite `Limit_Param[DM4310]`. The helper in `tests/_common.py` does this automatically:

```
pmax = mc.read_motor_param(motor, DM_variable.PMAX)
vmax = mc.read_motor_param(motor, DM_variable.VMAX)
tmax = mc.read_motor_param(motor, DM_variable.TMAX)
mc.Limit_Param[int(DM_Motor_Type.DM4310)] = [pmax, vmax, tmax]
```

6.5 Secondary issues found

1. **time.sleep bug (now fixed)** – `DM.CAN.py` previously had `from time import sleep` but called `time.sleep(0.001)` inside `control_Pos_Vel`, which raises `NameError: name 'time' is not defined` the first time position-speed mode is used. Patched by adding `import time`.
2. **Position-Speed v_{des}** is the trapezoidal cruise speed; if it is left at 0 the motor will not move. The student's loop uses `control_Pos_Vel(Motor1, q*8, 30)` which is fine.
3. $K_d = 0$ in **MIT position mode** causes oscillation/loss of control (PDF note). When using `controlMIT` for positioning, keep $K_d > 0$ (e.g. 0.5–2 for J4310).

4. **Communication-loss timeout:** if no CAN frame is received within the configured TIMEOUT the driver auto-disables. Send at least one command per timeout period.
5. **Bit-shift width on numpy.uint16 – q_uint >> 8** is performed on a numpy.uint16; this is correct but a plain int cast is safer if the type ever changes.

7 Recommended quick-start in MIT mode (DM-J4310-2EC)

```
m = Motor(DM_Motor_Type.DM4310, SlaveID=0x01, MasterID=0x11)
mc = MotorControl(serial.Serial('/dev/ttyACM0', 921600, timeout=0.5))
mc.addMotor(m)

# Sync library limits with firmware-stored ones (CRITICAL for J-series)
pmax = mc.read_motor_param(m, DM_variable.PMAX)
vmax = mc.read_motor_param(m, DM_variable.VMAX)
tmax = mc.read_motor_param(m, DM_variable.TMAX)
mc.Limit_Param[int(DM_Motor_Type.DM4310)] = [pmax, vmax, tmax]

mc.switchControlMode(m, Control_Type.MIT)
mc.enable(m)
mc.set_zero_position(m) # define current pose as 0 rad
mc.controlMIT(m, kp=30, kd=1.0, q=1.57, dq=0, tau=0) # 90 deg
```

8 Recommended quick-start in Position-Speed mode

```
mc.switchControlMode(m, Control_Type.POS_VEL)
mc.enable(m)
mc.control_Pos_Vel(m, P_desired=1.57, V_desired=5.0) # rad, rad/s
```

9 Control modes – full reference with Python examples

All commanded and feedback values are in **output-shaft units** (rad, rad/s, N·m). The 10:1 gearbox is transparent to the user.

9.1 Position-Speed mode (CAN_ID = 0x100 + ESC_ID)

What it does. Three-loop cascade (position → speed → current). **p_des** is the target position; **v_des** caps the trapezoidal cruise speed. The motor generates its own acceleration/deceleration profile.

Best for: deterministic point-to-point moves, robot joint positioning, pick-and-place.

```
# 03_pos_vel_control.py -- move to 90 deg and return
import math, time
from _common import open_bus
from DM_CAN import Control_Type

with open_bus(mode=Control_Type.POS_VEL, set_zero=True) as bus:
    for _ in range(400):
        bus.write("goal_pos_vel", {"j1": math.pi/2}, dq_des=5.0)
        time.sleep(0.01)
    for _ in range(400):
        bus.write("goal_pos_vel", {"j1": 0.0}, dq_des=5.0)
        time.sleep(0.01)
```

Key parameters:

- Acc / Dec (set in firmware): shape the ramp; increase if oscillation appears at the end of movement.
- $v_{des} = 0$ prevents movement – always set a non-zero cruise speed.

9.2 Speed mode (CAN_ID = 0x200 + ESC_ID)

What it does. Single-loop velocity control. The motor maintains the commanded speed. Position is ignored.

Best for: continuous rotation, conveyor-like tasks, proportional joystick control.

```
# 04_speed_control.py -- spin +5 rad/s then reverse
import time
from _common import open_bus
from DM_CAN import Control_Type

with open_bus(mode=Control_Type.VEL) as bus:
    for vel in [5.0, 0.0, -5.0, 0.0]:
        for _ in range(200):
            bus.write("goal_speed", {"j1": vel})
            time.sleep(0.01)
```

9.3 MIT mode (CAN_ID = ESC_ID) – three sub-modes

The MIT frame sends five scalars in 8 bytes. The motor computes:

$$\tau_{\text{ref}} = K_p(q_{\text{des}} - q_{\text{meas}}) + K_d(\dot{q}_{\text{des}} - \dot{q}_{\text{meas}}) + \tau_{\text{ff}}$$

By choosing which parameters are non-zero you get three distinct sub-modes.

9.3.1 MIT Position control ($K_p > 0$, $K_d > 0$)

Best for: tight position hold, fast response, back-drivability tuning.

```
# 05_mit_position.py
import math, time
from _common import open_bus

KP, KD = 30.0, 1.0      # Kp in [0,500], Kd in [0,5]
with open_bus(set_zero=True) as bus:
    for target in [math.pi/2, 0.0, -math.pi/4, 0.0]:
        for _ in range(300):
            bus.write("goal_position", {"j1": target}, kp=KP, kd=KD)
            time.sleep(0.01)
```

Tuning:

- Increase K_p for stiffer hold; decrease if the joint vibrates.
- Increase K_d to damp oscillations; too high feels sluggish.
- **Never set $K_d = 0$ when commanding position** – see PDF note.

9.3.2 MIT Velocity control ($K_p = 0, K_d > 0$)

Best for: smooth continuous rotation where precise position is not needed.

```
# 06_mit_velocity.py
import time
from _common import open_bus

KD = 1.0
with open_bus() as bus:
    for dq in [3.0, 0.0, -3.0, 0.0]:
        for _ in range(200):
            bus.write("goal_velocity", {"j1": dq}, kd=KD)
            time.sleep(0.01)
```

9.3.3 MIT Torque control ($K_p = 0, K_d = 0, \tau_{ff} \neq 0$)

Best for: force/impedance control, compliant tasks. Position and velocity are fully uncontrolled – the motor will accelerate if unloaded. Always use a mechanical stop or external safety check.

```
# 07_mit_torque.py
import time
from _common import open_bus

TAU = 0.5 # N.m -- keep small for unloaded tests
with open_bus() as bus:
    t0 = time.time()
    while time.time() - t0 < 1.5:
        bus.write("goal_torque", {"j1": TAU})
        time.sleep(0.01)
    # Damp to stop before disable
    for _ in range(100):
        bus.write("goal_velocity", {"j1": 0.0}, kd=1.0)
        time.sleep(0.01)
```

9.4 Mode comparison

Mode	CAN ID	Position?	Speed?	Tuning
Position-Speed	0x100+ID		via ramp	None
Speed	0x200+ID	–		None
MIT Position	ID		damped	K_p, K_d
MIT Velocity	ID	–		K_d
MIT Torque	ID	–	–	–

10 Impedance control and kinesthetic teaching

10.1 Concept

Impedance control makes the joint behave like a *virtual spring-damper*:

$$\tau_{\text{cmd}} = K_p(q_{\text{des}} - q_{\text{meas}}) + K_d(\dot{q}_{\text{des}} - \dot{q}_{\text{meas}}) + \tau_{\text{ff}}$$

By lowering K_p and K_d , the joint becomes *compliant*: an external force (your hand) can move it, and the motor applies only a gentle restoring force. This is the mechanism behind **kinesthetic teaching / teleoperation recording** used in imitation learning (e.g. ACT, DiffPolicy, UMI).

10.2 Physical interpretation of K_p and K_d

- K_p ($\text{N}\cdot\text{rad}^{-1}$): virtual spring stiffness. $K_p = 0$ means the joint is completely free. $K_p = 30$ – light spring, easy to push. $K_p = 300$ – stiff, hard to backdrive.
- K_d ($\text{N}\cdot\text{s}\cdot\text{rad}^{-1}$): virtual damping. Low K_d (< 0.2) feels frictionless but may vibrate. $K_d \approx 0.5$ – 1.0 feels smooth.

10.3 Recipe: “gravity-compensation + hand guiding”

1. Switch to MIT mode. Enable the motor.
2. Set $q_{\text{des}} = q_{\text{meas}}$ (current position) so the spring force is initially zero.
3. Set low K_p (e.g. 5–15) and low K_d (e.g. 0.3–0.8).
4. Add a τ_{ff} term that compensates gravity if the joint carries a load.
5. Record q_{meas} at each timestep – this is your demonstration.

```
"""08_kinesthetic_record.py -- move the motor by hand and record
    trajectory."""
import time, csv
from _common import open_bus

KP, KD = 5.0, 0.5
RECORD_HZ = 100

trajectory = []
with open_bus() as bus:
    bus.refresh()                                # prime state cache
    q_hold = bus.read_state()["j1"][0]           # start at current position
    print("Move the motor by hand. Press Ctrl+C to finish.")
    t0 = time.time()
    try:
        while True:
            q, dq, tau = bus.read_state()["j1"]
            t = time.time() - t0
            q_hold = q
            bus.write("mit_command", {"j1": q_hold}, kp=KP, kd=KD)
            trajectory.append([t, q, dq, tau])
            time.sleep(1.0 / RECORD_HZ)
    except KeyboardInterrupt:
        print(f"\nRecorded {len(trajectory)} samples.")

with open("demo_trajectory.csv", "w", newline="") as f:
    csv.writer(f).writerows(["t", "q", "dq", "tau"] + trajectory)
print("Saved demo_trajectory.csv")
```

This script is provided in the repo as `tests/08_kinesthetic_record.py`.

10.4 Spring-return to origin (`tests/10_impedance_spring_return.py`)

A companion script that uses MIT position mode with moderate stiffness to pull the joint back to zero after a demonstration:

```
# 10_impedance_spring_return.py
import time
```

```

from _common import open_bus

ORIGIN, KP, KD = 0.0, 20.0, 0.8
with open_bus(set_zero=True) as bus:
    print("Returning to origin. Press Ctrl+C to stop.")
    try:
        while True:
            q, dq, tau = bus.read_state()["j1"]
            bus.write("goal_position", {"j1": ORIGIN}, kp=KP, kd=KD)
            time.sleep(0.01)
    except KeyboardInterrupt:
        pass

```

10.5 Replaying a demonstration (imitation learning data collection)

After recording, load the CSV and replay:

```

# 09_replay_trajectory.py
import csv, time
from _common import open_bus

with open_bus() as bus:
    with open("demo_trajectory.csv") as f:
        rows = list(csv.DictReader(f))
        dt = float(rows[1]["t"]) - float(rows[0]["t"])
        for row in rows:
            q = float(row["q"])
            bus.write("goal_position", {"j1": q}, kp=30.0, kd=1.0)
            time.sleep(dt)

```

10.6 Parameter guidelines for different tasks

Task	K_p	K_d	Notes
Kinesthetic teaching (hand guiding)	0–10	0.3–0.8	Soft; operator moves joint
Gravity compensation only	0	0.5–1.0	Float; τ_{ff} holds weight
Impedance tracking (light spring)	10–30	0.5–1.5	Pushable but returns
Stiff position hold	100–300	1–3	Hard to backdrive
Replay / execution	30–80	1–2	Balance accuracy vs compliance

10.7 Safety considerations

- During hand guiding, always have an emergency-stop key (see **Ctrl+C** handler) or a hardware cut-off on the 24 V rail.
- $K_p = 0, K_d = 0$ (pure torque) will accelerate if $\tau_{ff} \neq 0$ – only use with mechanical limits or while holding the link.
- Keep communication rate ≥ 100 Hz. If the control loop drops below ~ 50 Hz the apparent damping decreases and oscillations may appear.
- The motor's overcurrent limit is 9.8 A; do not command torques that exceed peak torque (7 N·m at the output shaft).

11 Useful references in this repo

- docs/DM-J4310-en.pdf – the manual that this note summarises.
- src/DM_CAN.py – vendor driver (unchanged). Mirror at `motor-control-routine/Python/u2can/DM_CAN`.
- src/motor_config.py – single source of truth for hardware wiring (`DamiaoMotorConfig`, `DamiaoBusConfig`, `CubeMarsMotorConfig`, `CubeMarsBusConfig`, `DEFAULT_BENCH_CONFIG`).
- src/damiao_bus.py – LeRobot-style `DamiaoBus` wrapper.
- src/cubemars_bus.py – `CubemarsMotorsBus` for CubeMars AK-series (same bus contract as `DamiaoBus`).
- src/_common.py – test helpers: `open_bus()` context manager (tests 03–10), `open_motor()` + `safe_disable_close()` (legacy tests 01–02 only).
- tests/ – numbered test scripts 01–10 (see Table 1).
- motor-control-routine/ – C, C#, MATLAB, ROS, STM32 examples.
- old/ – archived root-level files (not active).

Table 1: Test scripts overview

File	Mode	Purpose
01_read_params.py	UART	Dump all firmware params
02_write_params.py	UART	Edit one param with confirmation
03_pos_vel_control.py	Pos-Speed	Point-to-point move
04_speed_control.py	Speed	Constant speed spin
05_mit_position.py	MIT Pos	Kp/Kd position hold
06_mit_velocity.py	MIT Vel	Velocity tracking
07_mit_torque.py	MIT Torque	Open-loop torque
08_kinesthetic_record.py	MIT	Hand guiding + CSV record
09_replay_trajectory.py	MIT	Replay recorded demo
10_impedance_spring_return.py	MIT	Spring return to origin